

La Classification

Apprendre à catégoriser - Pas à pas

Joseph Azar

Maître de conférences

Université Marie et Louis Pasteur

ELC2 · Introduction au machine learning · Séance 2



Commençons par un jeu...

Imaginez que vous montrez des photos à un enfant:



"Ça c'est un chat" → 10 photos de chats



"Ça c'est un chien" → 10 photos de chiens

Ensuite, vous montrez une nouvelle photo...

L'enfant peut dire si c'est un chat ou un chien! 🎯



C'est exactement ça, la **classification**!

Apprendre à partir d'exemples pour ranger dans des catégories

Qu'est-ce que la Classification?

Définition: La classification est une technique d'apprentissage supervisé qui prédit une **catégorie** (classe) pour une nouvelle donnée

Objectif

Prédire une **catégorie discrète**

Ex: chat/chien, spam/normal, malade/sain

Différence avec régression

Régression → valeur continue (prix, température)

Classification → catégorie (oui/non, A/B/C)

Plan de la présentation

1. 🤔 Qu'est-ce que la classification?
2. 🌐 Cas d'usage dans la vie réelle
3. 📊 Les types de classification
4. ♟️ Calcul manuel étape par étape
5. 🎯 K-Nearest Neighbors (KNN) - Simple!
6. 📈 Régression Logistique
7. 🌳 Arbres de décision
8. 📏 Mesurer les performances
9. 🕒 Les idées à connaître avant de coder



Applications dans la vie réelle

Email & Sécurité

- Spam ou pas spam?
- Fraude bancaire?
- Reconnaissance faciale

Médecine

- Cancer bénin ou malin?
- Diagnostic de maladies
- Classification de symptômes

Multimédia

- Reconnaissance d'images
- Classification de texte
- Analyse de sentiments

Business

- Client va acheter?
- Prêt: risqué ou pas?
- Produit défectueux?



Les types de classification

1 Classification Binaire (2 classes)

Oui/Non, Vrai/Faux, 0/1

Ex: Email spam? (oui/non) | Patient malade? (oui/non)

2 Classification Multi-classe (>2 classes)

Plusieurs catégories possibles

Ex: Chiffre manuscrit (0-9) | Type de fleur (iris, rose, tulipe)

3 Classification Multi-label

Plusieurs étiquettes en même temps

Ex: Photo avec chat ET chien | Film (action ET romance)

Exemple: Classifier des fruits

Objectif: Distinguer pommes  et oranges 

Fruit	Poids (g)	Diamètre (cm)	Couleur (0-100)	Classe
1	150	7	10	Pomme 
2	170	7.5	15	Pomme 
3	140	8	85	Orange 
4	130	7.5	90	Orange 
?	145	7.2	12	?

 Comment décider? On va voir plusieurs méthodes!



KNN - L'algorithme le plus simple!

Idée: "Dis-moi qui sont tes voisins, je te dirai qui tu es"

Principe en 3 étapes:

1. Trouver les K points les plus **proches** du nouveau point
2. Regarder leurs **classes**
3. Choisir la classe **majoritaire**

Exemple: $K=3$, je regarde mes 3 voisins les plus proches

Si 2 sont des pommes et 1 est une orange → C'est une pomme!



KNN - Calculons ensemble!



Nouveau fruit: Poids=145g, Diamètre=7.2cm, Couleur=12

K = 3 (on regarde les 3 voisins les plus proches)

Étape 1: Calculer les distances

$$\text{Distance} = \sqrt{[(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2]}$$

Fruit	Calcul distance	Distance	Classe
1	$\sqrt{[(150-145)^2 + (7-7.2)^2 + (10-12)^2]}$	5.39	Pomme 🍏
2	$\sqrt{[(170-145)^2 + (7.5-7.2)^2 + (15-12)^2]}$	25.18	Pomme 🍏
3	$\sqrt{[(140-145)^2 + (8-7.2)^2 + (85-12)^2]}$	73.18	Orange 🍊
4	$\sqrt{[(130-145)^2 + (7.5-7.2)^2 + (90-12)^2]}$	79.43	Orange 🍊

KNN - Le verdict!

Étape 2: Trier par distance (du plus proche au plus loin)

1. Fruit 1: 5.39 → 🍏 Pomme
2. Fruit 2: 25.18 → 🍏 Pomme
3. Fruit 3: 73.18 → 🍊 Orange
4. Fruit 4: 79.43 → 🍊 Orange

Étape 3: K=3, on garde les 3 plus proches

- ✅ Pomme (distance 5.39)
- ✅ Pomme (distance 25.18)
- ✅ Orange (distance 73.18)

Vote majoritaire: 2 pommes vs 1 orange

 **Résultat: C'est une POMME!** 🍏

À vous de jouer! KNN

 **Exercice:** Animal domestique ou sauvage?

Animal	Taille (cm)	Poids (kg)	Type
Chat	30	4	Domestique 🏠
Chien	50	15	Domestique 🏠
Loup	80	35	Sauvage 🌲
Renard	60	7	Sauvage 🌲
?	40	8	?

Calculez avec K=3 et trouvez la classe!

💡 Réponse: Distances → Chat(10.8), Chien(12.2), Renard(20.0), Loup(48.3)

K=3 les plus proches: Chat, Chien, Renard → 2 domestiques, 1 sauvage

✅ **Domestique** 🏠



La Régression Logistique

Idée: Calculer la **probabilité** d'appartenir à une classe



Contrairement à son nom, c'est pour la **classification**, pas la régression!

Elle donne: $P(\text{classe}=1)$ = probabilité entre 0 et 1

$$P(y=1) = 1 / (1 + e^{(-z)})$$

$$\text{où } z = w_1x_1 + w_2x_2 + \dots + b$$

Exemple:

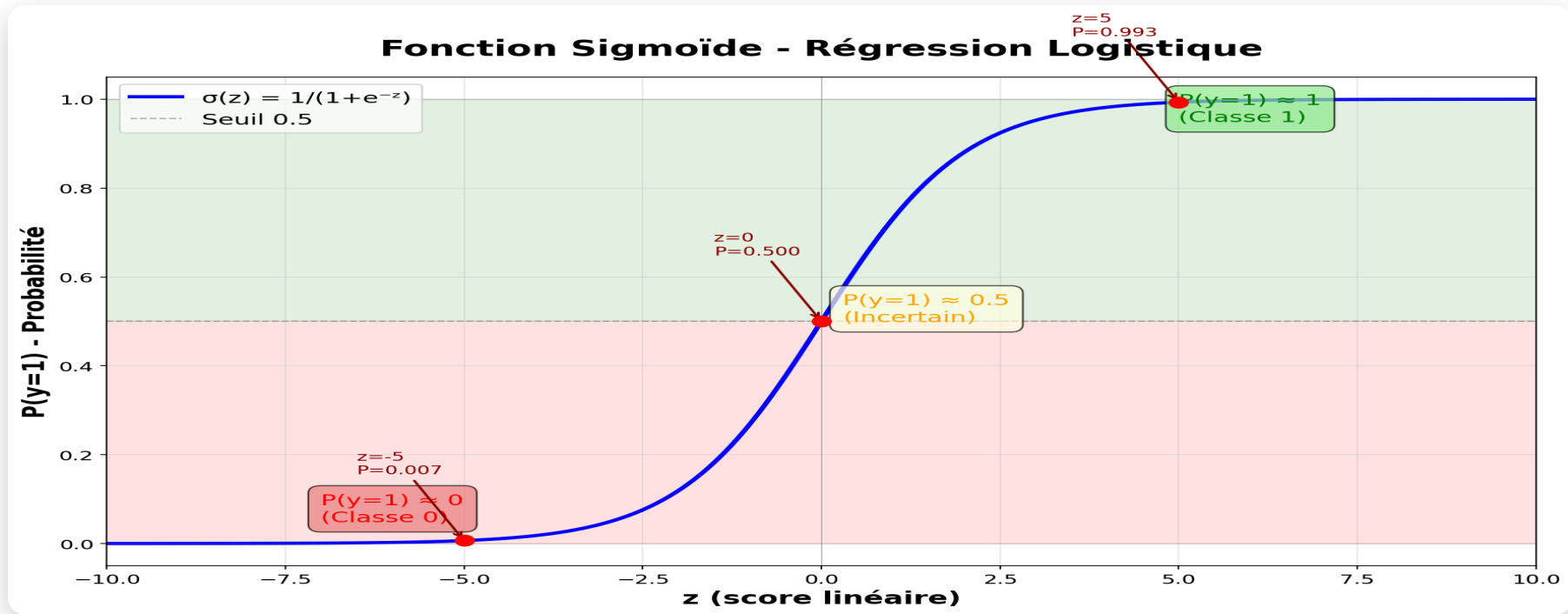
$P(\text{spam}) = 0.85 \rightarrow 85\%$ de chances que ce soit un spam ✓

$P(\text{spam}) = 0.15 \rightarrow 15\%$ de chances $\rightarrow$ C'est normal ✓



La fonction Sigmoid

La sigmoïde transforme n'importe quel nombre en probabilité (0 à 1)



💡 Interprétation:

- Si z très négatif ($z < -5$) → $P \approx 0$ (classe 0) ●
- Si $z \approx 0$ → $P \approx 0.5$ (incertain) ●
- Si z très positif ($z > 5$) → $P \approx 1$ (classe 1) ●



Régression Logistique - Calcul

Problème: Prédire si un email est un spam

Variables: Nb_mots_suspects (x_1), Nb_majuscules (x_2)

Modèle trouvé: $z = 2 \times x_1 + 1.5 \times x_2 - 3$

Email 1: 5 mots suspects, 10 majuscules

$$z = 2 \times 5 + 1.5 \times 10 - 3 = 10 + 15 - 3 = 22$$

$$P(\text{spam}) = 1 / (1 + e^{(-22)}) \approx \mathbf{0.9999999997} \approx 100\%$$

✅ **C'est un SPAM!**



Régression Logistique - Calcul

Problème: Prédire si un email est un spam

Variables: Nb_mots_suspects (x_1), Nb_majuscules (x_2)

Modèle trouvé: $z = 2 \times x_1 + 1.5 \times x_2 - 3$

Email 2: 0 mots suspects, 2 majuscules

$$z = 2 \times 0 + 1.5 \times 2 - 3 = 0 + 3 - 3 = 0$$

$$P(\text{spam}) = 1 / (1 + e^0) = 1/2 = \mathbf{0.5} = 50\%$$

⚠ **Incertain!** (seuil souvent à 0.5)

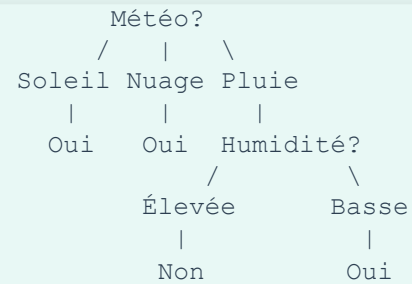


Les Arbres de Décision

Idée: Poser une série de **questions** pour arriver à la décision



Exemple: Jouer au tennis aujourd'hui?



Avantages: Facile à comprendre et interpréter!

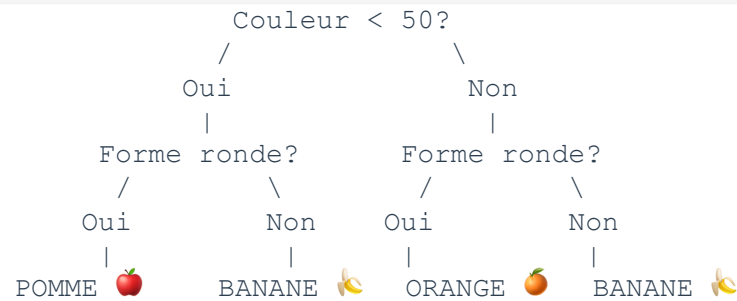


Chaque branche = une règle de décision



Arbre de Décision - Exemple Fruits

Classifier: Pomme 🍏 vs Orange 🍊 vs Banane 🍌



🎯 **Nouveau fruit:** Couleur=20, Forme=ronde

Q1: Couleur < 50? → Oui (va à gauche)

Q2: Forme ronde? → Oui (va à gauche)

✅ **Résultat: POMME 🍏**

À vous! Arbre de Décision

 Créez un arbre pour: Acheter un jeu vidéo?

Prix	Note /10	Multijoueur	Acheter?
€20	9	Oui	Oui ✓
€60	5	Oui	Non ✗
€30	8	Non	Oui ✓
€70	9	Oui	Non ✗

 Règle observée:

Si Prix > €50 → Non ✗

Sinon, si Note $\geq$ 8 → Oui ✓

Sinon → Non ✗



Quel algorithme choisir?

Algorithme	Avantages	Inconvénients	Quand l'utiliser?
KNN	Très simple Pas d'entraînement	Lent sur gros datasets Sensible à l'échelle	Petits datasets Peu de features
Régression Logistique	Rapide Donne probabilités	Seulement linéaire	Relations linéaires Interprétabilité
Arbres de Décision	Facile à interpréter Non-linéaire	Sur-ajustement Instables	Règles métier Données mixtes



Comment mesurer la performance?

Matrice de Confusion - Le tableau de bord

		Prédiction	
		Positif	Négatif
Réal	Positif	VP Vrai Positif	FN Faux Négatif
	Négatif	FP Faux Positif	VN Vrai Négatif

VP: On dit chat, c'était un chat ✓

VN: On dit chien, c'était un chien ✓

FP: On dit chat, mais c'était un chien ✗

FN: On dit chien, mais c'était un chat ✗



Les métriques importantes

$$\text{Accuracy (Exactitude)} = (\text{VP} + \text{VN}) / \text{Total}$$

% de prédictions correctes

$$\text{Precision (Précision)} = \text{VP} / (\text{VP} + \text{FP})$$

Quand je dis "positif", à quelle fréquence j'ai raison?

$$\text{Recall (Rappel)} = \text{VP} / (\text{VP} + \text{FN})$$

Des vrais positifs, combien ai-je trouvé?

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Équilibre entre Precision et Recall



Calcul des Métriques - Exemple

Détection de spam: 100 emails testés

	Prédit Spam	Prédit Normal
Vrai Spam	40 (VP)	10 (FN)
Vrai Normal	5 (FP)	45 (VN)

$$\text{Accuracy} = (40+45)/100 = 85\% \checkmark$$

$$\text{Precision} = 40/(40+5) = 88.9\% \text{ (spam détecté = souvent vrai)}$$

$$\text{Recall} = 40/(40+10) = 80\% \text{ (80\% des spams trouvés)}$$

$$\text{F1-Score} = 2 \times (0.889 \times 0.8) / (0.889 + 0.8) = 84.2\%$$



Avant de coder : cinq idées à bien comprendre

Elles reviennent dans tous les projets de machine learning. Pas de formule : des exemples de la vie courante.

1. **Découper** les données : le train et le test
2. La **fuite de données** : le test qui triche sans le dire
3. Le **sous-apprentissage** et le **sur-apprentissage**
4. Les données **déséquilibrées**
5. **Deux classes** ou plusieurs ?



Découper les données : pourquoi ?

Un étudiant révise avec **20 exercices corrigés**.

À l'examen, on lui redonne **les mêmes 20 exercices** : il a 20/20.

Est-il bon ? On ne peut pas le savoir : il a peut-être seulement **retenu les réponses**.

Un vrai examen pose des **questions nouvelles**. Pour un modèle, c'est pareil : ce qui compte, c'est qu'il réussisse sur des **clients qu'il n'a jamais vus**.

On cache donc une partie des données : le modèle ne la voit pas pendant qu'il apprend.



Découper les données : comment ?

Exemple : 100 clients à répartir

80 clients pour apprendre : le **train**

20 : le **test**

1. **Mélanger** les clients au hasard
2. **Mettre de côté** 20 % : c'est le **test**. On n'y touche plus
3. **Apprendre** sur les 80 % restants : c'est le **train**
4. **Juger** le modèle une seule fois, sur le test

💡 **Mêmes proportions partout** : si 1 client sur 4 résilie, on veut 1 sur 4 dans le train **et** 1 sur 4 dans le test.



La fuite de données (1/2) : le test qui triche

Fuite de données : le modèle reçoit, sans qu'on s'en aperçoive, des informations qui viennent du test.

C'est comme si le professeur laissait **le sujet de l'examen sur la table** pendant les révisions : la note sera belle, mais elle ne prouve rien.

❌ Mauvais

1. Calculer l'âge moyen avec **tous** les clients, test compris
 2. Remplir les âges manquants du train avec cette moyenne
- Le test a déjà influencé la préparation.

✅ Correct

1. **Découper** d'abord
 2. Calculer l'âge moyen sur le **train** seulement
 3. Remplir le train **et** le test avec cette même moyenne
- Le test sert uniquement à juger.



La fuite de données (2/2) : la réponse cachée

Comtoise Télécom veut prédire quels clients vont **résilier**. Le fichier contient une colonne « **motif de résiliation** ».

Client	Motif de résiliation	A résilié ?
A	Prix trop élevé	Oui
B	(vide)	Non
C	Offre d'un concurrent	Oui
D	(vide)	Non

Le motif n'existe que pour les clients **déjà partis**. Le modèle n'a plus rien à deviner : **100 % de réussite**... et il est inutile, car au moment de décider, le client n'est pas encore parti.

Règle : une colonne n'a le droit d'aider que si on la connaît **avant** l'événement à prédire. Un score trop beau pour être vrai doit toujours faire douter.



Sous-apprentissage et sur-apprentissage

Trois étudiants préparent le même examen. On compare leur note aux **exercices** (qu'ils ont vus) et à l'**examen** (des questions nouvelles).

Étudiant	Comment il révise	Exercices (train)	Examen (test)	Verdict
Léa	Révise à peine	8 / 20	7 / 20	Sous-apprentissage : elle n'a pas assez appris
Tom	Apprend les corrigés par cœur	20 / 20	8 / 20	Sur-apprentissage : il a retenu, il n'a pas compris
Inès	Comprend le cours	16 / 20	15 / 20	Bon apprentissage

Tom a la meilleure note aux exercices, mais l'écart avec l'examen est énorme. C'est pour cela qu'on juge un modèle **sur le test**.

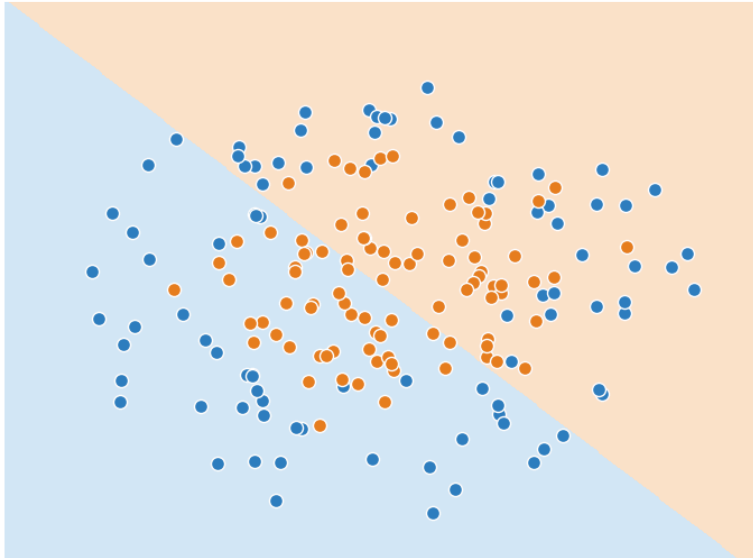


Sous- et sur-apprentissage : en image

Les mêmes clients, trois modèles. Les deux couleurs sont deux catégories ; le fond montre ce que le modèle prédit.

Trop simple

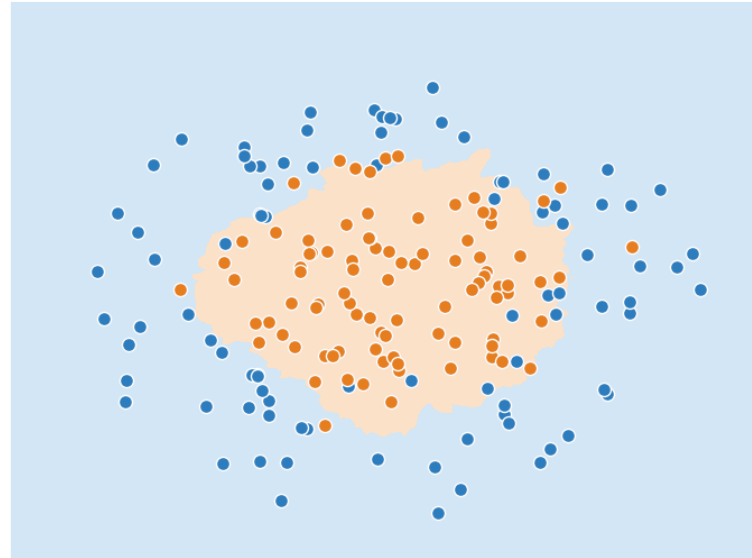
il ne pose qu'une seule règle



exercices (train) : 51%
examen (test) : 52%

Bien équilibré

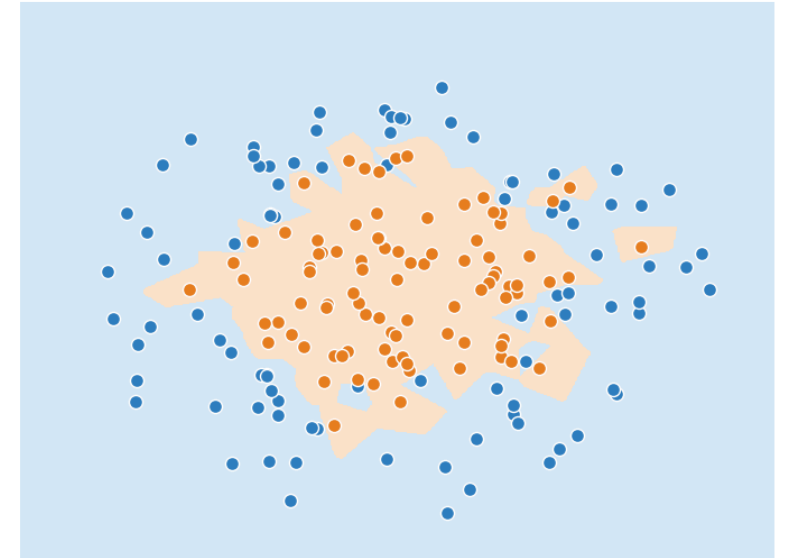
il suit la forme générale



exercices (train) : 89%
examen (test) : 88%

Appris par cœur

il retient chaque exemple



exercices (train) : 100%
examen (test) : 80%



Comment repérer le problème ?

Comparez toujours **deux notes** : celle sur le **train** (les exercices) et celle sur le **test** (l'examen).

Note sur le train	Note sur le test	Diagnostic
basse	basse	sous-apprentissage : le modèle n'a pas compris
très haute	nettement plus basse	sur-apprentissage : il a appris par cœur
haute	presque la même	bon apprentissage

💡 Un modèle évalué seulement sur le train aura toujours l'air bon. D'où l'intérêt du test.



Sous- et sur-apprentissage : que faire ?

Sous-apprentissage

Le modèle est **trop simple**.

- Lui donner plus de moyens (un arbre avec plus de questions)
- Lui donner de **meilleures colonnes**

Sur-apprentissage

Le modèle est **trop compliqué**.

- Le simplifier (un arbre avec moins de questions)
- Lui donner **plus d'exemples**

Avec KNN : le nombre de voisins K règle la difficulté.

- **K = 1** : chaque client ne regarde que son voisin le plus proche. Le modèle colle à chaque exemple : **par cœur**
- **K très grand** : on vote presque toujours comme la majorité. Le modèle est **trop simple**



Données déséquilibrées : le piège de la bonne note

Une banque veut repérer les **fraudes**. Sur 1 000 paiements, **10** seulement sont frauduleux.

990 paiements normaux

• 10 fraudes

Un modèle paresseux répond « normal » à tous les paiements, sans rien regarder.

Bonnes réponses : 990 sur 1 000 = **99 %**

Fraudes trouvées : **0 sur 10**

99 % de réussite, et le modèle ne sert à rien. La bonne note vient seulement du fait que la catégorie « normal » est très fréquente.



Données déséquilibrées : que faire ?

Ça arrive souvent : fraude bancaire, maladie rare, panne de machine... et dans notre fichier de clients : 1 sur 4 résilie, donc « personne ne part » fait déjà **74 %**.

1. **Ne pas se fier à l'accuracy seule.** Regarder aussi le *recall* (combien de fraudes ai-je trouvées ?) et la *precision* (mes alertes sont-elles justes ?)
2. **Se comparer au modèle paresseux.** S'il fait 99 % et que « tout est normal » fait aussi 99 %, le modèle n'a rien appris
3. **Garder les mêmes proportions** dans le train et dans le test
4. Parfois : **collecter plus d'exemples rares**, ou donner plus d'importance aux erreurs sur la catégorie rare

Deux classes ou plusieurs ?

2 classes : binaire

- spam / pas spam
- le client reste / résilie
- malade / sain

Plus de 2 classes : multi-classe

- pomme / orange / banane
- un chiffre écrit à la main (10 classes)
- la race d'un chien

Quel impact ? Le hasard ne donne pas la même note (si les classes sont à égalité) :

	2 classes	3 classes	10 classes
En répondant au hasard	1 juste sur 2 : 50 %	1 sur 3 : 33 %	1 sur 10 : 10 %
Un score de 60 % est...	faible	bon	très bon

Un score se lit toujours **par rapport au hasard** (ou au modèle paresseux), jamais seul.

Plus de deux classes : la matrice grandit

Un modèle trie **30 fruits** (10 pommes, 10 oranges, 10 bananes) :

	Prédit : pomme	Prédit : orange	Prédit : banane
Réel : pomme	8	2	0
Réel : orange	3	6	1
Réel : banane	0	1	9

En vert, la **diagonale** = les bonnes réponses : $8 + 6 + 9 = 23$ sur 30, soit **77 %**

En rouge, les autres cases disent **qui est confondu avec qui** : ici, 3 oranges prises pour des pommes

On lit la réussite **classe par classe** : pommes retrouvées 8 sur 10, oranges 6 sur 10, bananes 9 sur 10

Avec 2 classes, la matrice a 4 cases ; avec 10 classes, 100. *Precision* et *recall* se calculent alors classe par classe (« les pommes contre tout le reste », puis « les oranges contre tout le reste »...), puis on fait la moyenne.



Les cinq idées en une phrase

Idée	À retenir
Train / test	On apprend sur une partie des données et on juge sur une autre, que le modèle n'a jamais vue
Fuite de données	Le test ne doit rien apprendre au modèle : ni par la préparation, ni par une colonne « trop belle »
Sous / sur-apprentissage	Trop simple : il rate tout. Par cœur : parfait sur le train, mauvais sur le test. Comparez les deux notes
Données déséquilibrées	Une bonne accuracy peut cacher un modèle inutile : regardez le recall et la precision
2 classes ou plus	Le hasard n'est pas à 50 % partout, et la matrice de confusion grandit avec le nombre de classes



Code Python - KNN

```
# Importer les bibliothèques
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Données d'entraînement (fruits)
X_train = np.array([
    [150, 7, 10], # Pomme
    [170, 7.5, 15], # Pomme
    [140, 8, 85], # Orange
    [130, 7.5, 90] # Orange
])
y_train = np.array(['Pomme', 'Pomme', 'Orange', 'Orange'])

# Créer le modèle KNN avec K=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Prédire un nouveau fruit
nouveau_fruit = np.array([[145, 7.2, 12]])
prediction = knn.predict(nouveau_fruit)
print(f"C'est une: {prediction[0]}")

# Voir les probabilités
probas = knn.predict_proba(nouveau_fruit)
print(f"Probabilités: {probas}")
```



Code Python - Régression Logistique

```
# Régression Logistique pour spam
from sklearn.linear_model import LogisticRegression

# Données: [mots_suspects, majuscules]
X_train = np.array([
    [0, 2], # Normal
    [1, 5], # Normal
    [5, 10], # Spam
    [7, 15] # Spam
])
y_train = np.array([0, 0, 1, 1]) # 0=Normal, 1=Spam

# Créer et entraîner
model = LogisticRegression()
model.fit(X_train, y_train)

# Prédire
email_test = np.array([[3, 8]])
prediction = model.predict(email_test)
probabilite = model.predict_proba(email_test)

print(f"Classe: {prediction[0]}")
print(f"Probabilité spam: {probabilite[0][1]:.2%}")
```



Code Python - Arbre de Décision

```
# Arbre de Décision
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt

# Données
X_train = np.array([[20, 9], [60, 5], [30, 8], [70, 9]])
y_train = np.array(['Acheter', 'Pas acheter', 'Acheter', 'Pas acheter'])

# Créer l'arbre
clf = DecisionTreeClassifier(max_depth=3)
clf.fit(X_train, y_train)

# Visualiser l'arbre
plt.figure(figsize=(12,8))
tree.plot_tree(clf, feature_names=['Prix', 'Note'],
               class_names=['Acheter', 'Pas acheter'],
               filled=True)
plt.show()

# Prédire
nouveau_jeu = np.array([[40, 7]])
print(f"Décision: {clf.predict(nouveau_jeu)[0]}")
```



Points clés à retenir

1. 🎯 Classification = prédire une **catégorie**
2. 👤 KNN = Vote des voisins (simple et intuitif)
3. 📈 Régression Logistique = Probabilités (0 à 1)
4. 🌳 Arbres = Série de questions/règles
5. 📊 Toujours évaluer avec matrice de confusion
6. ⚖️ Choisir selon le problème et les données
7. 🧪 Tester sur données séparées!

Pièges à éviter

Classes déséquilibrées

99% classe A, 1% classe B → 99% accuracy facile mais inutile!

→ Utiliser F1-Score, pas seulement Accuracy

Sur-ajustement

Arbre trop profond = parfait sur train, mauvais sur test

→ Limiter la profondeur, utiliser validation croisée

Oublier la normalisation (KNN)

Feature 1: [0-1], Feature 2: [0-1000] → Distance biaisée!

→ Toujours normaliser pour KNN



Conclusion

"La classification, c'est comme apprendre à un ordinateur à reconnaître des motifs, comme nous le faisons naturellement!"

Vous savez maintenant:

-  Comprendre la classification
-  Appliquer KNN, Régression Logistique, Arbres
-  Calculer distances et probabilités
-  Évaluer avec métriques appropriées
-  Coder en Python avec scikit-learn



Prêts à classifier le monde!